

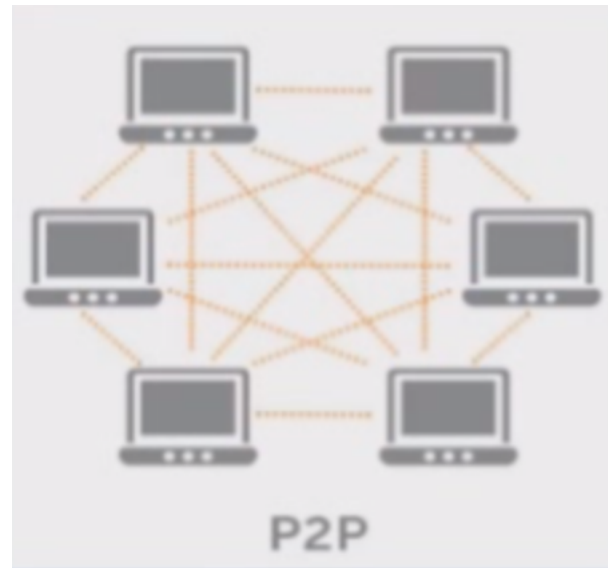
交易的传播与验证

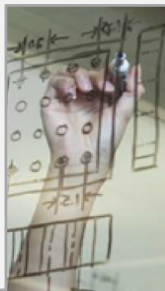
交易-区块-区块链

Transaction –Block-Blockchain

交易的传播与验证

- 交易包含两部分：n输入和m输出， $n \geq 0$ ， $m > 0$
 - 输入：要被花费的UTXO + 解锁脚本 [私钥签名，公钥]
 - 输出：一个或多个UTXO（币值 + 锁定脚本[公钥的哈希]）
- 钱包软件生成交易，并向邻居节点传播
 - 节点对收到的交易进行验证，并丢弃不合法交易
 - 交易的size要小于区块size的上限
 - 交易输入UTXO是存在的（全节点UTXO数据库检索）
 - 交易输入UTXO没有被其他交易引用，防止双花（Double Spending）
 - 输入总金额 $\geq$ 输出总金额，解锁脚本验证
- 将合法的交易加入本地Transaction数据库中，并发送给临近节点





区块生成与链接

区块的生成

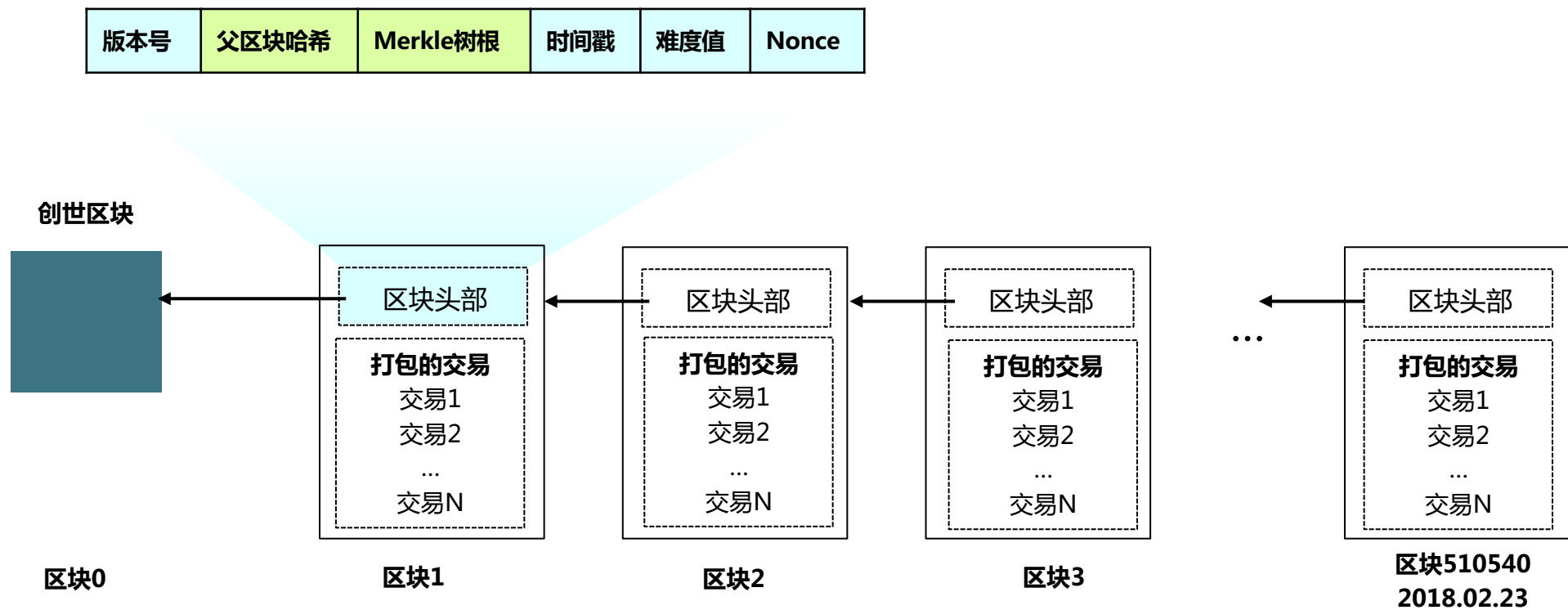
- 矿工在挖矿前要组建区块
 - 将coinbase交易打包进区块
 - 将交易池中高优先级的交易打包进区块
 - 优先级=交易额度*UTXO深度/交易size （类比银行）
 - 交易size：防止粉尘攻击 UTXO深度：防止频繁转账攻击

- 创建区块头部

版本号	父区块哈希	Merkle树根	时间戳	难度值	Nonce
-----	-------	----------	-----	-----	-------

- 挖矿成功后，将计算出来的随机数nonce填入区块头部，向邻近节点传播 （接力赛）

区块链



区块0存储的信息：

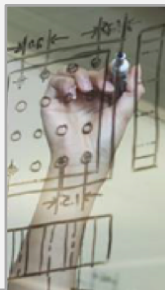
当时正是英国财政大臣第二次出手疏解银行危机之时。 表示对中心化银行和政府的不满。

区块的验证与链接

- **相邻节点收到新区块后，立即做如下检查**
 - 验证POW的nonce值是否符合难度值
 - 检查时间戳是否小于当前时间2小时 ???
 - 检查Merkle树根是否正确
 - 检查区块size要小于区块size上限
 - 第一笔交易必须是coinbase交易
 - 验证每个交易
 - 比特币代码网站 http://btc.yt/lxr/satoshi/ident?_i=CheckBlock

区块验证源代码

```
1333 bool CBlock::CheckBlock() const
1334 {
1335     // These are checks that are independent of context
1336     // that can be verified before saving an orphan block.
1337
1338     // Size limits
1339     if (vtx.empty() || vtx.size() > MAX_BLOCK_SIZE || ::GetSerializeSize(*this, SER_NETWORK) > MAX_BLOCK_SIZE)
1340         return DoS(100, error("CheckBlock() : size limits failed"));
1341
1342     // Check proof of work matches claimed amount
1343     if (!CheckProofOfWork(GetHash(), nBits))
1344         return DoS(50, error("CheckBlock() : proof of work failed"));
1345
1346     // Check timestamp
1347     if (GetBlockTime() > GetAdjustedTime() + 2 * 60 * 60)
1348         return error("CheckBlock() : block timestamp too far in the future");
1349
1350     // First transaction must be coinbase, the rest must not be
1351     if (vtx.empty() || !vtx[0].IsCoinBase())
1352         return DoS(100, error("CheckBlock() : first tx is not coinbase"));
1353     for (int i = 1; i < vtx.size(); i++)
1354         if (vtx[i].IsCoinBase())
1355             return DoS(100, error("CheckBlock() : more than one coinbase"));
1356
1357     // Check transactions
1358     BOOST_FOREACH(const CTransaction& tx, vtx)
1359         if (!tx.CheckTransaction())
1360             return DoS(tx.nDoS, error("CheckBlock() : CheckTransaction failed"));
1361
1362     // Check that it's not full of nonstandard transactions
1363     if (GetSigOpCount() > MAX_BLOCK_SIGOPS)
1364         return DoS(100, error("CheckBlock() : out-of-bounds SigOpCount"));
1365
1366     // Check merkle root
1367     if (hashMerkleRoot != BuildMerkleTree())
1368         return DoS(100, error("CheckBlock() : hashMerkleRoot mismatch"));
1369
1370     return true;
1371 }
```

Merkle树

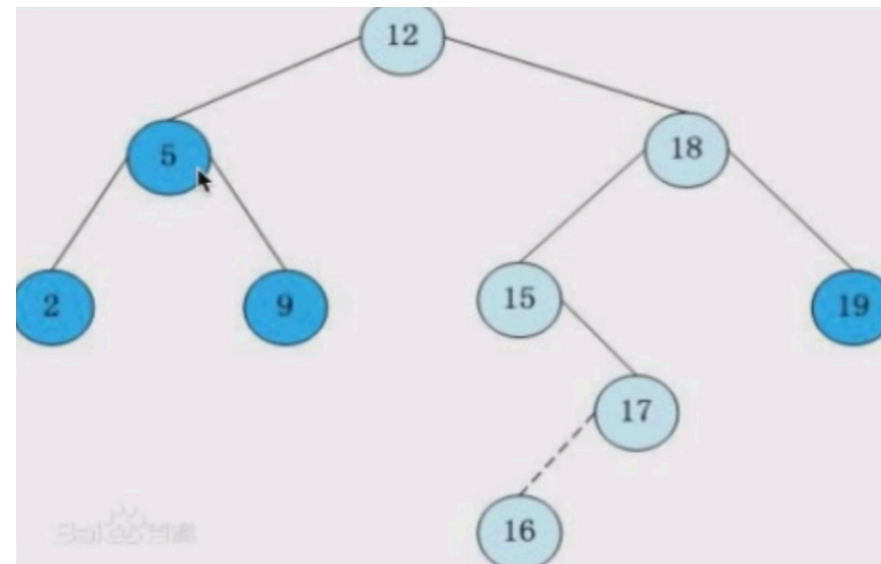
Merkle树结构

- **树：由多个节点组成的一种数据结构**

- 每个节点存储数据
- 节点分类：根节点 Root，父节点，子节点，兄弟节点

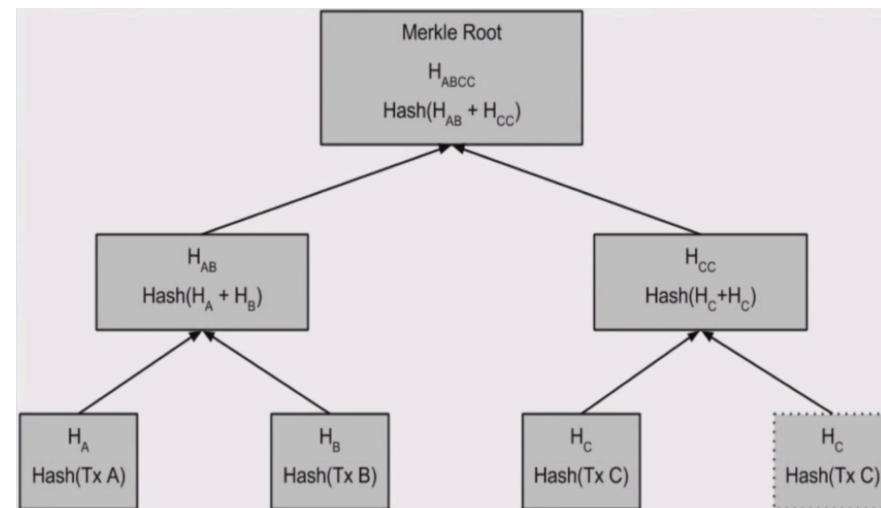
- **特殊树：搜索二叉树（方便查找）**

- 12 5 2 18 19 15 17 16 9



- **Merkle树作用**

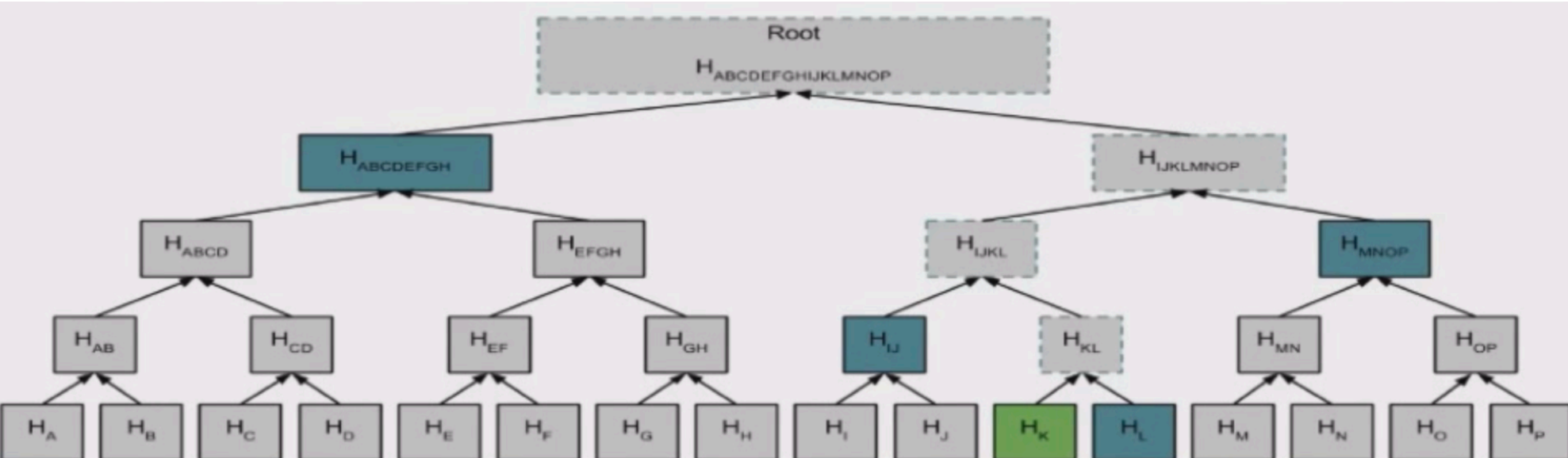
- 防止数据篡改（节点存储Hash值）
- 快速验证某个交易是否存在
- 从叶子节点构造树



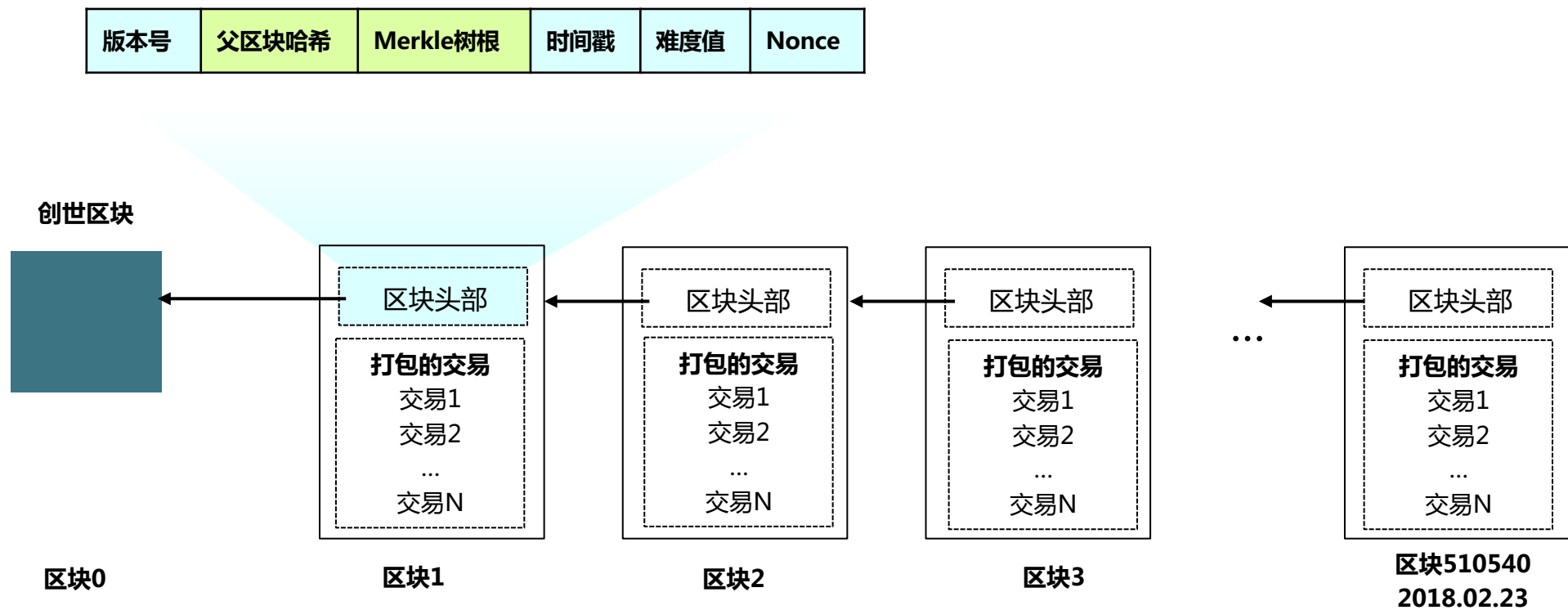
Merkle Path 验证路径 （怎么验证交易是否在区块里面）

- 只需 $\log_2 N$ 个已知数，即可验证K是否被包含在区块中
- 若要验证交易K是否在区块里，只需要 $\log_2 16=4$ 步，需知道交易 H_L H_{IJ} H_{MNOP} $H_{ABCDEFGH}$

交易数量（笔）	区块大小	路径步数（Hashes）	路径大小（Bytes）
16	4KB	4 hashes	128B
512	128KB	9 hashes	288B
2048	512KB	11 hashes	352B
65535	16M	16 hashes	512B

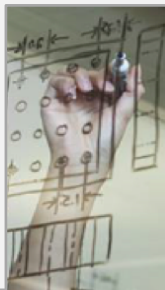


区块链



区块0存储的信息：

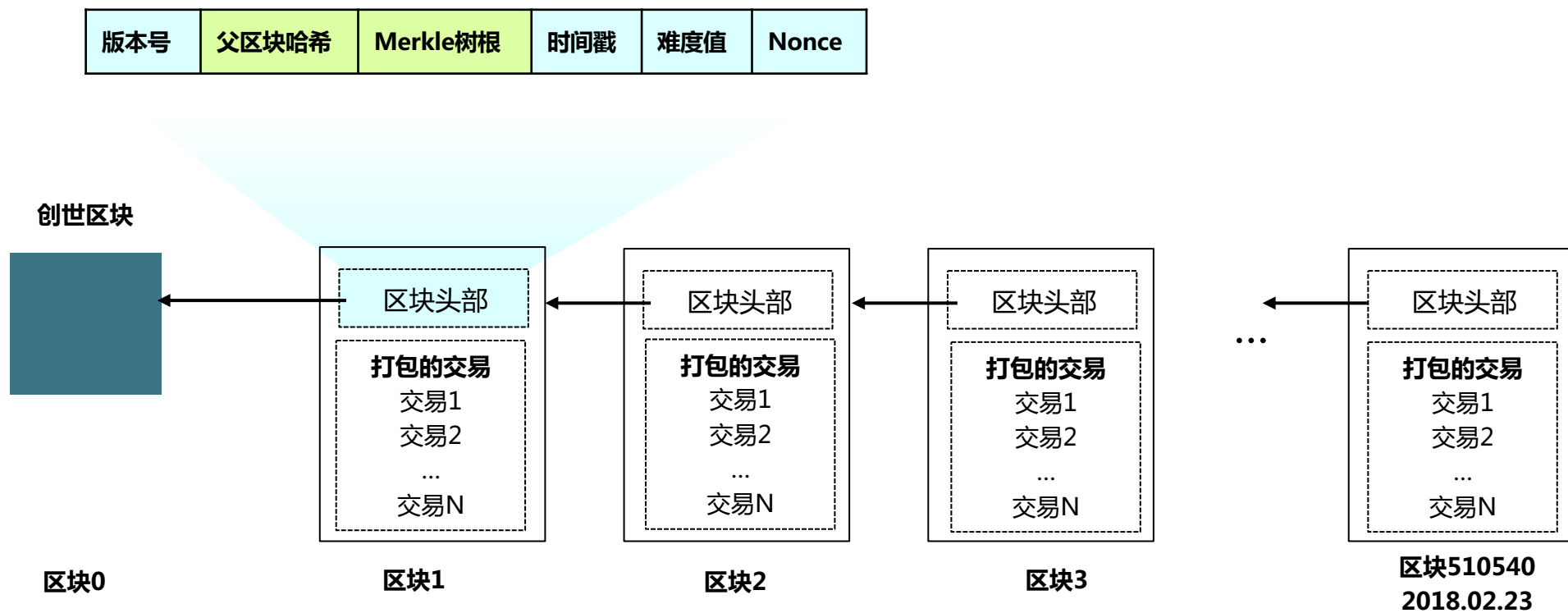
当时正是英国财政大臣第二次出手疏解银行危机之时。 表示对中心化银行和政府的不满。



哈希在区块链中应用总结

哈希算法在区块链中应用总结

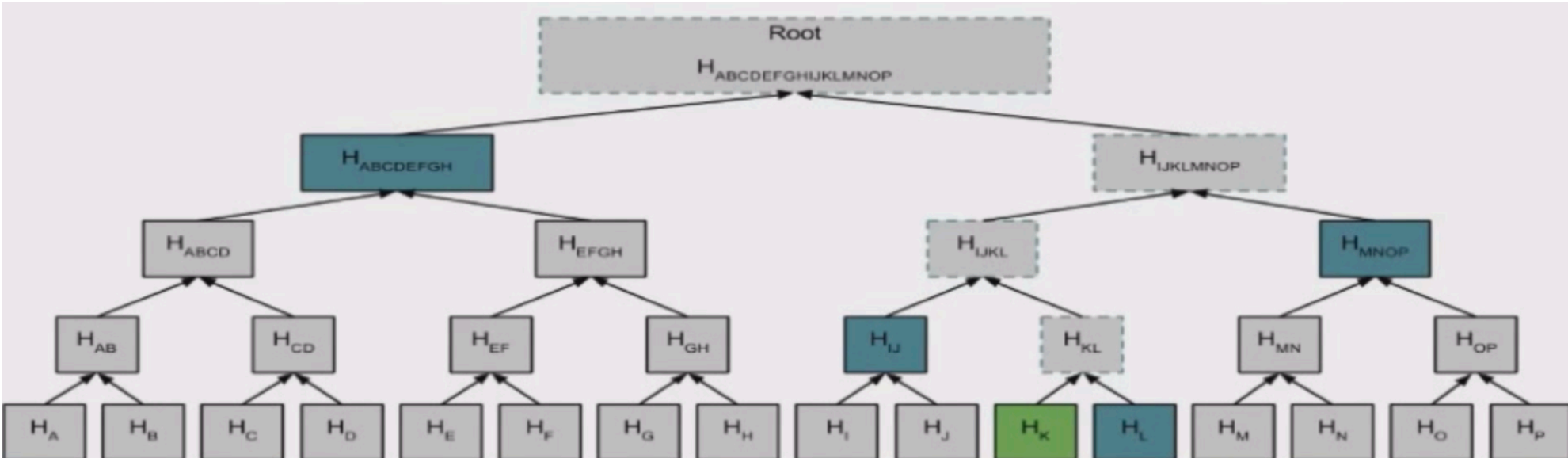
- **Merkle树根**：区块链中对每笔交易哈希，防止交易篡改
- **父区块哈希**：防止区块链整体被篡改
- **比特币地址**：公钥的哈希值



Merkle Path 验证路径 （怎么验证交易是否在区块里面）

- 只需 $\log_2 N$ 个已知数，即可验证K是否被包含在区块中
- 若要验证交易K是否在区块里，只需要 $\log_2 16 = 4$ 步，需知道交易 H_L H_{IJ} H_{MNOP} $H_{ABCDEFGH}$

交易数量（笔）	区块大小	路径步数（Hashes）	路径大小（Bytes）
16	4KB	4 hashes	128B
512	128KB	9 hashes	288B
2048	512KB	11 hashes	352B
65535	16M	16 hashes	512B



| THANKS 「授人以渔」